

CO2

ASSESSMENT TOOL 3



NAME

V.MAHALAKSHMI



REG NO.

192521181



COURSE

CRYPTOGRAPHY AND NETWORK SECURITY



COURSE CODE

CSA5113

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES

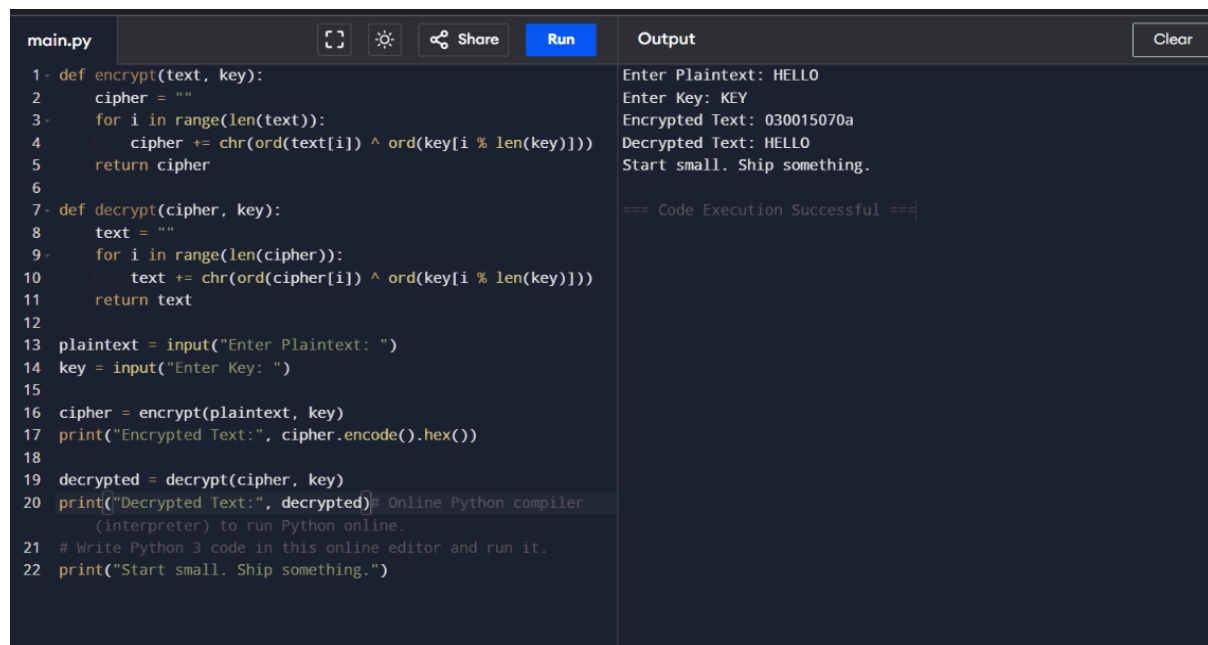
Implementation)

A Python program implementing DES and 3DES encryption produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).

C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.



The screenshot shows an online Python compiler interface. On the left, a code editor displays a Python script named 'main.py'. The script defines two functions, 'encrypt' and 'decrypt', which perform a simple XOR-based encryption and decryption using a key. The main logic takes user input for plaintext and key, encrypts the plaintext, and then decrypts the resulting ciphertext. Comments in the code refer to an 'Online Python compiler (interpreter) to run Python online.' and instruct the user to 'Write Python 3 code in this online editor and run it.' The right side of the interface shows the 'Output' pane, which contains the execution results: 'Enter Plaintext: HELLO', 'Enter Key: KEY', 'Encrypted Text: 030015070a', 'Decrypted Text: HELLO', and 'Start small. Ship something.' Below the output, a status message reads '=== Code Execution Successful ==='.

```
main.py
1- def encrypt(text, key):
2-     cipher = ""
3-     for i in range(len(text)):
4-         cipher += chr(ord(text[i]) ^ ord(key[i % len(key)]))
5-     return cipher
6-
7- def decrypt(cipher, key):
8-     text = ""
9-     for i in range(len(cipher)):
10-        text += chr(ord(cipher[i]) ^ ord(key[i % len(key)]))
11-    return text
12-
13- plaintext = input("Enter Plaintext: ")
14- key = input("Enter Key: ")
15-
16- cipher = encrypt(plaintext, key)
17- print("Encrypted Text:", cipher.encode().hex())
18-
19- decrypted = decrypt(cipher, key)
20- print("Decrypted Text:", decrypted) # Online Python compiler
    (interpreter) to run Python online.
21- # Write Python 3 code in this online editor and run it.
22- print("Start small. Ship something.")
```

Output

Enter Plaintext: HELLO
Enter Key: KEY
Encrypted Text: 030015070a
Decrypted Text: HELLO
Start small. Ship something.

=== Code Execution Successful ===

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements AES-like block cipher modes (ECB, CBC, CFB, OFB) but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.

B. Refactor the code to minimize redundant computations and memory usage.

C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

```

main.py  [Icons] [Share] [Run] [Clear]
4 result = ""
5 for i in range(len(text)):
6     result += chr(ord(text[i]) ^ ord(key[i % len(key)]))
7 return result
8
9 plaintext = input("Enter Plain Text: ")
10 key = input("Enter Key: ")
11
12 # ECB Mode
13 ecb = xor_encrypt(plaintext, key)
14 print("\nECB Encrypted (Hex):", ecb.encode().hex())
15
16 # CBC Mode Simulation
17 iv = "A" * len(key)
18 cbc_input = ""
19 for i in range(len(plaintext)):
20     c = chr(ord(plaintext[i]) ^ ord(iv[i % len(iv)]))
21     cbc_input += c
22
23 cbc = xor_encrypt(cbc_input, key)
24 print("CBC Encrypted (Hex):", cbc.encode().hex())
25
26 # Decryption
27 cbc_decrypt = xor_encrypt(cbc, key)
28 original = ""
29 for i in range(len(cbc_decrypt)):

```

```

^ Enter Plain Text: HELLO
  Enter Key: KET123

  ECB Encrypted (Hex): 0300187d7d
  CBC Encrypted (Hex): 4241593c3c
  Decrypted Text: HELLO

  === Code Execution Successful ===

```

3. Debugging RSA Cryptosystem Implementation

A Python-based RSA encryption system fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.

B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).

C. Evaluate the trade-offs between key size, security, and computational performance.

```

main.py  [Icons] [Share] [Run] [Clear]
15 q = 13
16
17 n = p * q
18 phi = (p - 1) * (q - 1)
19
20 # Public Key
21 e = 7
22 while gcd(e, phi) != 1:
23     e += 1
24
25 # Private Key
26 d = mod_inverse(e, phi)
27
28 print("Public Key (e,n):", (e, n))
29 print("Private Key (d,n):", (d, n))
30
31 # Input
32 msg = int(input("Enter Message (Number): "))
33
34 # Encryption
35 cipher = (msg ** e) % n
36 print("Encrypted Message:", cipher)
37
38 # Decryption
39 plain = (cipher ** d) % n
40 print("Decrypted Message:", plain)

```

```

^ Public Key (e,n): (7, 143)
  Private Key (d,n): (103, 143)
  Enter Message (Number): 9
  Encrypted Message: 48
  Decrypted Message: 9

  === Code Execution Successful ===

```

A Python implementation of the Diffie-Hellman key exchange is vulnerable to performance bottlenecks and potential security flaws.

B. Optimize exponentiation operations for large prime values.

C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

```
main.py  [Icons] [Share] [Run] [Output] [Clear]

3 # Public values
4 p = 23      # Prime Number
5 g = 5       # Primitive Root
6
7 # Private Keys
8 a = int(input("Enter Private Key of User A: "))
9 b = int(input("Enter Private Key of User B: "))
10
11 # Public Keys
12 A = (g ** a) % p
13 B = (g ** b) % p
14
15 print("Public Key of User A:", A)
16 print("Public Key of User B:", B)
17
18 # Shared Secret Key
19 keyA = (B ** a) % p
20 keyB = (A ** b) % p
21
22 print("Shared Key for User A:", keyA)
23 print("Shared Key for User B:", keyB)
24
25 if keyA == keyB:
26     print("Key Exchange Successful")
27 else:
28     print("Key Exchange Failed")
```

```
Enter Private Key of User A: 6
Enter Private Key of User B: 15
Public Key of User A: 8
Public Key of User B: 19
Shared Key for User A: 2
Shared Key for User B: 2
Key Exchange Successful

=== Code Execution Successful ===
```

A Python script implementing Elliptic Curve Cryptography for key exchange produces incorrect results for certain curve parameters.

main.py

 Share

Run

```
3 # Curve Parameters
4 p = 17
5 g = 3
6
7 # Private Keys
8 a = int(input("Enter Private Key of User A: "))
9 b = int(input("Enter Private Key of User B: "))
10
11 # Public Keys
12 PA = (g ** a) % p
13 PB = (g ** b) % p
14
15 print("Public Key of User A:", PA)
16 print("Public Key of User B:", PB)
17
18 # Shared Secret
19 KA = (PB ** a) % p
20 KB = (PA ** b) % p
21
22 print("Shared Key for User A:", KA)
23 print("Shared Key for User B:", KB)
24
25 if KA == KB:
26     print("Key Exchange Successful")
27 else:
28     print("Key Exchange Failed")
```

Output

Clear

Enter Private Key of User A: 4
Enter Private Key of User B: 8
Public Key of User A: 12
Public Key of User B: 7
Shared Key for User A: 11
Shared Key for User B: 11
Key Exchange Successful

=== Code Execution Successful ===